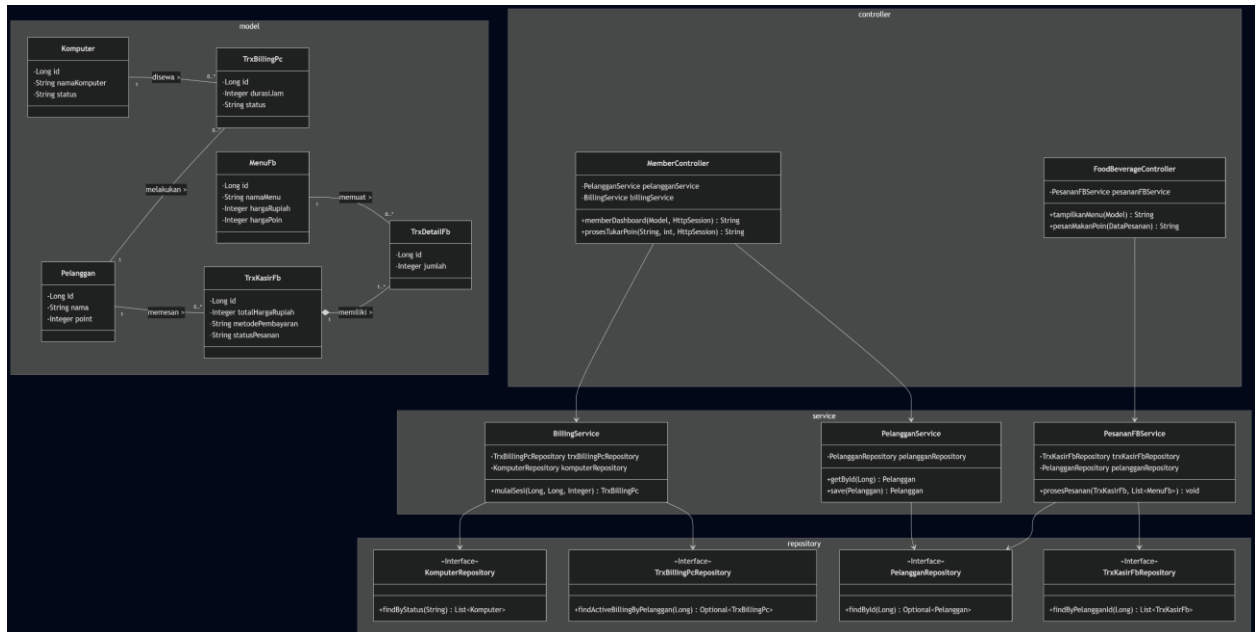


Class Diagram - NexaNet Cafe

1. Skema Class Diagram Keseluruhan

Berikut ini adalah skema *class diagram* dalam sistem NexaNet Cafe yang mencakup struktur arsitektur *backend* secara utuh, mulai dari *Controller*, *Service*, *Repository*, hingga *Model (Entity)*.



2. Penjelasan Lengkap Class Diagram Proyek

Arsitektur kode program NexaNet Cafe mengimplementasikan *Layered Architecture* (Arsitektur Berlapis) di dalam ekosistem Spring Boot. Pemisahan kelas ke dalam paket-paket (*packages*) ini bertujuan untuk memenuhi prinsip *Separation of Concerns* (SoC) agar setiap lapis memiliki tanggung jawab yang terisolasi dan modular.

Berikut adalah analisis fungsional mendalam untuk masing-masing komponen:

A. Lapisan Model (Domain Entities)

Lapis ini berisi representasi objek bisnis (*Plain Old Java Object* / POJO) yang dipetakan langsung ke tabel *database* menggunakan anotasi JPA/Hibernate.

- **Entitas Inti:** Pelanggan, Komputer, TrxBillingPc, MenuFb, TrxKasirFb, dan TrxDetailFb bertindak sebagai entitas utama di dalam memori.
- **Objek Referensi:** Berbeda dengan ERD yang menyimpan relasi berupa kolom ID asing (*Foreign Key*), pada *Class Diagram* relasi ini diwujudkan dalam bentuk Objek Referensi langsung. Contohnya, kelas TrxKasirFb akan memiliki atribut bertipe objek Pelanggan pelanggan, bukan sekadar Long pelanggan_id.

B. Lapisan Repositori (Data Access Layer)

Semua kelas di dalam paket ini berupa antarmuka (*interface*) yang mewarisi JpaRepository dari Spring Data JPA.

- Lapis ini tidak memerlukan penulisan implementasi *query* SQL manual untuk operasi CRUD (Create, Read, Update, Delete) standar karena sudah disediakan otomatis oleh kerangka kerja Spring.
- Repositori ini menyediakan metode khusus, seperti `findActiveBillingByPelanggan()` pada `TrxBillingPcRepository`, untuk melayani mekanisme pengecekan status penyewaan PC yang sedang berjalan.

C. Lapisan Service (Business Logic Layer)

Paket ini memuat kelas-kelas konkret tempat seluruh kalkulasi algoritma, logika operasional, serta penegakan validasi bisnis berjalan.

- **PelangganService:** Bertanggung jawab mengelola manipulasi data member, termasuk validasi penambahan atau pengurangan saldo poin (*Reward System*).
- **PesananFBService:** Mengkoordinasikan proses transaksi kantin. Kelas ini menyuntikkan (*inject*) beberapa dependensi sekaligus, seperti `TrxKasirFbRepository` dan `PelangganRepository`. Sebelum pesanan diproses, kelas ini bertugas memvalidasi kecukupan saldo poin pelanggan dan menghitung total harga sebelum menyimpannya ke pangkalan data.

D. Lapisan Controller (API View Layer)

Bertindak sebagai gerbang komunikasi eksternal aplikasi. Kelas-kelas di *layer* ini (seperti `MemberController` dan `FoodBeverageController`) bertugas menerima *HTTP Request* dari klien, mengarahkan pemrosesan ke *Service*, lalu mengembalikan data ke dalam wadah Thymeleaf Model (*View*) atau *Redirect* rute. *Layer* ini murni bertugas mengarahkan alur data dan dilarang memuat komputasi bisnis secara langsung.

3. Analisis Pola Hubungan Antar Kelas

Dalam diagram ini, terdapat dua jenis pola hubungan (*relationship*) utama yang menjaga arsitektur aplikasi tetap kokoh:

1. Asosiasi / Komposisi Domain (Hubungan Entitas)

Menunjukkan struktur data objek di memori. Contoh utamanya adalah kelas `TrxKasirFb` yang mengkomposisikan objek `TrxDetailFb`. Sebuah eksistensi detail pesanan tidak akan bermakna dan akan ikut musnah jika data induk pesannya (Header Kasir) tidak ada atau dihapus. Selain itu, `TrxBillingPc` juga berelasi kuat dengan Pelanggan dan Komputer karena sesi *billing* tidak mungkin berjalan tanpa adanya subjek penyewa dan objek PC.

2. Ketergantungan Aliran Komponen (*Dependency Injection*)

Ditunjukkan melalui tanda panah ($\rightarrow$) antar *layer*, ini merepresentasikan prinsip arsitektur *Loose Coupling* (Ketergantungan Rendah). Kelas di lapis atas selalu dan hanya bergantung pada kelas di lapis tepat di bawahnya:

- *Controller* menyuntikkan kelas *Service*.
- *Service* menyuntikkan kelas *Repository*.

Pola delegasi searah ini menjamin alur penelusuran kesalahan (*debugging*) menjadi lebih terprediksi, mempermudah pengujian unit (*unit testing*), serta menjaga integritas kode program tetap kokoh saat aplikasi NexaNet dikembangkan dalam skala yang lebih masif di masa depan.